Python Programming

COURSE EXERCISES

Exercise 1 : Python Tooling 3

Exercise 2 : Variables and Data Types 6

Exercise 3 : Data Structures 9

Exercise 4 : Control Flow 13

Exercise 5 : Functions 16

Exercise 6 : Modules 18

Exercise 7 : Comprehensions 20

Exercise 8 : Exception Handling 22

Exercise 9 : Python IO 24

Exercise 10 : Python Database Access 26

Exercise 11 : Python Classes 28

APPENDIX

Exercise 12 : Type Hinting 31

Exercise 13 : Python Libraries 34

Exercise 1 : Python Tooling

This exercise takes about fifteen minutes and leaves you with a working Python 3.14, the `uv` project manager, the `ruff` linter and an editor that understands Python. Everything you do in the rest of the course runs in the project you build here.

Step 1 : Install uv

`uv` is the project manager the course uses, and it can install Python itself, so it is the only thing you install by hand. On Windows, run this in PowerShell :

```
powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"
```

On macOS or Linux, run this in a terminal :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Close and reopen the terminal afterwards so that `uv` is on your `PATH`, then confirm it works with `uv --version`.

See also : https://docs.astral.sh/uv/

Step 2 : Install Python

Let `uv` fetch the interpreter the course uses :

```
uv python install 3.14
```

This needs no administrator rights and leaves any Python already on the machine alone. Confirm it with `uv python list`.

Step 3 : Install an editor

Install Visual Studio Code from `https://code.visualstudio.com`, then add two extensions from the Extensions panel, the square icon on the left :

- `Python`, published by Microsoft
- `Ruff`, published by Astral

Any editor works, but the course demonstrates VS Code and the slides assume it.

See also : https://docs.astral.sh/ruff/

Step 4 : Create your course project

You write your own code this week in a project of your own, the same way you would at work. From the folder where you keep your work :

```
uv init course-work
cd course-work
```

Open that folder in VS Code, then use the Command Palette, `Python: Select Interpreter`, and choose the interpreter in `.venv`.

Step 5 : Confirm the toolchain runs

Run each of these from inside `course-work` :

```
uv --version
uv run python --version      # prints Python 3.14.x
uv run ruff --version
```

Then write a first script. Create `hello.py` in the project :

```
def greet(name):
    return f"Hello, {name}"


print(greet("Python"))
```

Run it with `uv run python hello.py`. You should see `Hello, Python`.

Step 6 : See the linter work

Run the linter over your project :

```
uv run ruff check .
```

It should report nothing. Now break something on purpose. Add an unused import at the top of `hello.py` :

```
import json
```

Run `uv run ruff check .` again. It reports `F401 'json' imported but unused`. Notice that VS Code underlines the same line. Remove the import, or let the linter do it for you :

```
uv run ruff check . --fix
```

If time permits : look at `demos/00a-python-tooling/02_ruff_findings.py`, which is written badly on purpose. Run `uv run ruff check` over it and read every finding before you look at the comments explaining them.

Exercise 2 : Variables and Data Types

In this exercise you build the first version of the calculator application that you will extend throughout the course.

Step 1

Create a directory or project named `exercise2`. Next create a Python module `calculator2.py` in this directory and create two variables `nr1` and `nr2`. Assign integer values to them as follows :

```
nr1 = 16
nr2 = 23
```

Use your own values if you like. Print the values, the type and the memory location of the variables with :

```
print (nr1)
print (nr2)
print (type(nr1))
print (type(nr2))
print (id(nr1))
print (id(nr2))
```

You should see that the type of the variables is `<class 'int'>`. The `id` function gives some numeric value which actually indicates the memory address in a default Python implementation.

Now change the values of the variables by assigning them to a new value and print the memory location of the variables again with the `id` function. Notice that the id of the variables has changed just by assigning them. This illustrates the fact that integer variables are read-only : they cannot change once they are created. When you change an integer variable you actually create a new one.

Step 2

Next perform arithmetic operations on the variables by adding, subtracting, multiplying and dividing them. Store the result of these operations in other variables. An example for the addition operation is :

```
addresult = nr1 + nr2
```

Display the results of the operations with an explanatory message :

```
print ("The result of the addition of : " + str(nr1) + " and " +
str(nr2) + " is : ")
print (addresult)
```

Notice the conversion function `str` that turns an integer into a string, so that it can be added to other strings.

As an alternative you can also use the string format operator `%` for printing values in a string :

```
print ("The result of the addition of : %d and %d is : " % (nr1, nr2))
```

A third alternative is to use the new style of formatting in Python 3 with the `format()` function. Figure out using the course manual or documentation how to use this style of string formatting. And a fourth alternative would be to use an f-string. Figure that one out as well.

Also display the types of the results. The first three are of `<class 'int'>`. Notice that the type of the division is `<class 'float'>`.

Step 3

Now use the `input()` function to ask for the input of two numbers from the user as follows :

```
s1 = input("give me number 1 : ")
s2 = input("give me number 2 : ")
```

The `input()` function returns a string. Find out about the type of the return values by printing their type. Now perform addition with the `+` operator on the user input. With user input 1 and 3 the result will be 13, because 1 and 3 are seen as strings : the strings are concatenated.

```
addresult = s1 + s2  # s1='1', s2='3'
print (addresult)    # prints 13
```

Next convert the strings to integers with the `int()` conversion function as follows :

```
addresult = int(s1) + int(s2)
```

With the same input the result will now be 4. When the given input cannot be converted to an int, a runtime error occurs :

```
ValueError: invalid literal for int() with base 10: 'w'
```

Exercise 3 : Data Structures

In this exercise you extend the calculator with lists, multidimensional lists, tuples and dictionaries.

Step 1

Create a project or directory named `exercise3` and a Python module `calculator3.py`. Next create an empty list variable and ask the user for the input of two float numbers. Add these numbers, store the result in a variable and append the numbers and the result of their addition to the list. Repeat this operation until the user quits the program by pressing the red button in the IDE. Your code could look like the following :

```
reslist = []
while True:
    f1 = float(input("Give me number 1 : "))
    f2 = float(input("Give me number 2 : "))
    addresult = f1 + f2
    reslist.append(f1)
    reslist.append(f2)
    reslist.append(addresult)
```

Also print the content of the list, print the length of the list and print a subset of the list as follows :

```
print (f"Content of list : {reslist}")
print (f"Length of list : {len(reslist)}")
print (f"Part of list : {reslist[0:3]}")
```

Finally remove the first element of the list and print the list again :

```
print (reslist[0])
del reslist[0]
print (f"Content of list after deletion : {reslist}")
```

Step 2

Now create a multidimensional list in which each row contains a calculation with as elements the type of operation, the two float numbers and the result. Also calculate the results of the subtraction, multiplication and division and store these as rows in the multidimensional list. Start by allocating memory for the multidimensional list as follows :

```
multireslist = [[0,0,0,0], [0,0,0,0], [0,0,0,0], [0,0,0,0]]
```

Then store the results of the operations in separate lists and add these lists at appropriate places in the multidimensional list. For the add operation your code could look as follows :

```
addresult = f1 + f2
addlist = ["Add", f1, f2, addresult]
multireslist[0] = addlist
```

Write similar code for the subtract, multiply and divide operations. Print the multidimensional list after storing the results. Your result should look as follows :

```
Content of list : [['Add', 2.0, 2.0, 4.0], ['Subtract', 2.0, 2.0,
0.0],
['Multiply', 2.0, 2.0, 4.0], ['Divide', 2.0, 2.0, 1.0]]
```

Finally try to change some element in one of the sublists and notice that you are able to make the change. Then change one of the sublists to a tuple :

```
addlist = ("Add", f1, f2, addresult)
```

Notice that when you try to change elements of this tuple you get the following error :

```
TypeError: 'tuple' object does not support item assignment
```

Step 3

Now create an empty dictionary. Ask the user for the input of two numbers like before. Perform add, subtract, multiply and divide operations on the numbers. Store the numbers and the results of the calculation in a small list for each calculation type :

```
resdict = {}
while True:
    f1 = float(input("Give me number 1 : "))
    f2 = float(input("Give me number 2 : "))
    addresult = f1 + f2
    addlist = [f1, f2, addresult]
```

Now store the list in the dictionary under the key `"Add"` as follows :

```
resdict["Add"] = addlist
```

Use similar code for the other calculation operations and print the dictionary after the input of two numbers.

Try to read and extend the value at a dictionary key. The reading will succeed if the dictionary key was created before, but you will get an error if the key did not exist. Next make sure that the keys exist before starting to ask input from the user, by initializing the dictionary as follows :

```
resdict = {"Add": [], "Subtract": [], "Multiply": [], "Divide": []}
```

Now store multiple results of add calculations under the `"Add"` key as follows :

```
resdict["Add"] = resdict["Add"] + addlist
```

Step 4

Ask the user to input a list of integers and split the input string into a list :

```
s = input("Give me a set of numbers separated by space characters : ")
slist = s.split(' ')
print (slist)
```

Also sort and reverse the list with the `sort` and `reverse` functions. Finally convert the list of strings to a list of integers with :

```
intlist = []
for x in slist:
    x = int(x)
    intlist.append(x)
print (intlist)
```

Feel free to experiment with other string, list and dictionary functions. Use the documentation.

Exercise 4 : Control Flow

In this exercise the calculator becomes interactive : the user chooses the type of calculation.

Step 1

Create a project or directory named `exercise4` and a Python module `calculator4.py` . Copy the contents of step 3 of the previous exercise to `calculator4.py` .

Now ask the user, beside the numbers for the calculations, also which type of calculation the user wants to perform. Use `if` , `else` and `elif` statements to determine which branch of the code must be entered. If the user enters `"Quit"` the application should stop :

```
resdict = {"Add": [], "Subtract": [], "Multiply": [], "Divide": []}
while True:
    calc_action = input("What type of calculation do you want to do :
")
    if calc_action == "Quit":
        break
    else:
        f1 = float(input("Give me number 1 : "))
        f2 = float(input("Give me number 2 : "))
        if calc_action == "Add":
            addresult = f1 + f2
            addlist = [f1, f2, addresult]
            resdict["Add"] = resdict["Add"] + addlist
        elif calc_action == "Subtract":
            ...
```

Step 2

Now add an option to calculate the sum and average of a list of numbers. Copy part of the code from step 4 of exercise 3 to convert a list of strings to a list of integers. Based on the input of the desired calculation type, you will have to make a distinction between the input of a list of numbers or only two numbers :

```
elif (calc_action == "Sum") or (calc_action == "Average"):
    s = input("Give a set of numbers separated by space characters :
")
```

Now calculate the sum or the average of the list of numbers in a `for` loop. Do not use `sum` as a variable name since this is a reserved word; use `summation` instead. For the calculation of the sum the code could look as follows :

```
summation = 0
average = 0
if calc_action == "Sum":
    for x in slist:
        x = int(x)
        summation = summation + x
        intlist.append(x)
    print (f"Sum of list {intlist} is {summation}")
```

Step 3

Replace the `while True` statement at the beginning of the code by a test on the input of the calculation type. Stop the while loop when the input is `"Quit"` :

```
calc_action = "Continue"
while calc_action != "Quit":
    calc_action = input("What type of calculation do you want to do :
")
```

Also add an `else` statement to the while loop that gets executed when the while loop stops :

```
else:
    print ("Thank you for using our software")
```

Exercise 5 : Functions

In this exercise the inline calculations are replaced by functions.

Step 1

Create a project or directory named `exercise5` and a Python module `calculator5.py`. Copy the contents of the previous exercise to `calculator5.py`.

Write functions to add, subtract, multiply and divide two numbers and replace the code that does the calculations inline by a call to these functions :

```
def add(i, j):
    z = i + j
    return z
```

Calling this function is done with :

```
addresult = add(f1, f2)
```

Step 2

Write a function `convert_to_int_list` that converts a list of input characters to a list of integers :

```
def convert_to_int_list(slist):
    ilist = []
    for x in slist:
        x = int(x)
        ilist.append(x)
    return ilist
```

Call this function with as parameter the input list given by the user for the sum and average calculations.

Also write a function `summate` that takes a list of integers as an argument and returns the sum of the integers in that list. And write a function `average` that takes a list of integers and returns the average of these integers :

```
def summate(ilist):
    s = 0
    for x in ilist:
        s = s + x
    return s
```

Replace the inline summation and averaging code by calls to these functions.

Step 3

Add another calculation option to the calculator called `"Faculty"`. When the user enters this, one number should be entered and the factorial of this number should be calculated. Hereby the factorial of n is defined as n * (n-1) * (n-2) * (n-3) * ... * 1.

Create a recursive function called `faculty` to perform this calculation and call this function.

Exercise 6 : Modules

In this exercise the calculation functions are moved to their own module and package.

Step 1

Create a project or directory named `exercise6` and a Python module `calculator6.py`. Copy the contents of `calculator5.py` from the previous exercise to `calculator6.py`.

Next create a Python module `calculations.py`. Remove the function definitions from `calculator6.py` and put them into `calculations.py`. As a result all function calls in `calculator6.py` will give errors because they are calling undefined functions.

Resolve the errors by importing the calculations module with an import statement in the first line of `calculator6.py` :

```
import calculations
```

This is not enough. You will also need to prefix all function names with the module name where they come from :

```
addresult = calculations.add(f1, f2)
```

Step 2

It is inconvenient to put the module name as prefix before the function name. Therefore there is an alternative syntax called `from <module> import <name>`. Now change to this alternative and name all the function names that must be imported :

```
from calculations import faculty, convert_to_int_list, summate,
average, add, subtract, multiply, divide
```

As a result the prefixes before the method calls can be removed in `calculator6.py`.

Step 3

Finally make a `calcu` package. Put the `calculations.py` module in this package, edit the `__init__.py` file and import the functions from the package in `calculator6.py`.

Exercise 7 : Comprehensions

In this exercise functions are replaced by functional programming constructs like lambdas, comprehensions and `reduce`.

Step 1

Create a project or directory named `exercise7` and a Python module `calculator7.py`. Copy the contents of `calculator6.py` from the previous exercise to `calculator7.py`. Do not copy the `calcu` package.

Put the import `from functools import reduce` in the first line of `calculator7.py`.

Step 2

Replace the following code by a list comprehension :

```
s = input("Give a set of numbers separated by space characters : ")
slist = s.split(' ')
intlist = convert_to_int_list(slist)
```

We do not need a `convert_to_int_list` function because we can do the conversion in a one-liner using a list comprehension.

Step 3

Replace the following code by a call to the `reduce` function which uses a lambda function for summing :

```
summation = summate(intlist)
print (f"Sum of list {intlist} is {summation}")
```

Step 4

Replace the following code by a call to the `reduce` function which uses a lambda function for summing, and divide by the length of the list of integers :

```
av = average(intlist)
print (f"Average of list {intlist} is {av}")
```

We can calculate the average in a one-liner using the `reduce` function, a lambda and the length of the list with the `len` function.

Step 5

Replace the following code with one line which uses the definition and call of a lambda function for `add`. Do the same for `subtract`, `divide` and `multiply`. All can be replaced by one-liners which do not use an external function :

```
addresult = add(f1, f2)
addlist = [f1, f2, addresult]
resdict["Add"] = resdict["Add"] + addlist
```

Step 6

Finally replace the calculation of the factorial with the following code and observe that it gives the correct results :

```
n = int(input("Give a positive integer number : "))
fac = reduce((lambda x, y: x * y), list(range(1, n+1)))
```

Exercise 8 : Exception Handling

In this exercise the calculator is made robust against invalid input.

Step 1

Create a project or directory named `exercise8` and a Python module `calculator8.py`. Copy the contents of `calculator6.py` to `calculator8.py`. Also copy the `calcu` package which now should contain comprehensions.

Write `try: except:` statements for the following cases. In all these cases the current program crashes. With exception handling we can ask for new input.

1. Division by zero.
2. Wrong input for Add, Subtract, Multiply and Divide operations.
3. Wrong input for the number for the factorial.
4. Wrong input for the list of integers.

Step 2

Define an exception class. Then write a `try: except:` statement in which you throw and catch that specific exception. Use code from the demo directories or the internet as example code.

Step 3

The code that we have made up to now is not very clean. We should make our script more maintainable and extensible. One big script in which everything happens is not so good. The if statements are also nested too much.

Now refactor the code so it consists of a main from where three functions are called, namely : `OneNumberCalculations`, `TwoNumberCalculations` and `MultipleNumberCalculations`. Each of these functions should get a `calc_action` string as parameter.

Write this script in the file `calculatorclean.py`. Make sure the script works exactly the same as the non-clean version.

Exercise 9 : Python I0

In this exercise every calculation is logged to a text file.

Step 1

Create a project or directory named `exercise9` and a Python module `calculator9.py`. Copy the contents of `calculator8.py` from the previous exercise to `calculator9.py`. Also copy the `calcu` package.

Open a text file for writing just before entering the while loop that asks the user to choose a calculation type. Use a `with` statement, so that the file is closed however the loop ends, and indent the whole loop inside it :

```
with open('calculationdata.txt', mode='w', encoding='UTF-8') as fw:
    while calc_action != "Quit":
        ...
```

Next add the details of a calculation to a line in this file when each calculation is performed. For the Faculty calculation you could use the following code :

```
fw.write(f"{calc_action},{x},{faculty(x)}\n")
```

For the Sum and Average calculations you could use the following code :

```
fw.write(f"{calc_action},{intlist},{summate(intlist)}\n")
```

And for the Add, Subtract, Divide and Multiply calculations you could use the following code just after the input of the numbers :

```
fw.write(f"{calc_action},{f1},{f2}")
```

And the following code just after each calculation :

```
fw.write(f"{add(f1, f2)}\n")
```

There is no `fw.close()` to write. Leaving the `with` block closes the file, which is what makes the text appear on disk.

Now enter some calculations to test your code and observe the contents of the file.

Step 2

Next create a new Python module `readcalculations.py`, read the lines of the file and display its contents with the following code :

```
with open('calculationdata.txt', encoding='UTF-8') as fr:
    for line in fr:
        s = line.rstrip('\n')
        slist = s.split(',')
        print(slist)
```

Note two things: the file object is iterated directly rather than through `readlines()`, which avoids holding the whole file in memory, and `mode='r'` is left out because reading is the default.

The results could look as follows :

```
['Faculty', '12', '479001600']
['Sum', '[1', ' 1', ' 3]', '5']
['Average', '[4', ' 5', ' 6]', '5.0']
```

Exercise 10 : Python Database Access

In this exercise the file logging is replaced by a SQLite database.

Step 1

Create a project or directory named `exercise10` and a Python module `calculator10.py`. Copy the contents of `calculator9.py` from the previous exercise to `calculator10.py`. Also copy the `calcu` package. You will replace the file access code with database access code.

Before entering the while loop in which the user is asked to choose the type of calculation, use the following code to create a database `calculations.db` and a table `Calculations` :

```
con = sqlite3.connect('calculations.db')
with con:
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS Calculations")
    cur.execute("CREATE TABLE Calculations (Id VARCHAR(10), Nr1 INT,
Nr2 INT, Result DECIMAL)")
```

Next consider the Faculty calculation type and replace the code where data is written to the file with code that writes data to the database :

```
with con:
    cur = con.cursor()
    cur.execute("INSERT INTO Calculations ('Id', 'Nr1', 'Nr2',
'Result') VALUES(?,?,?,?)",
                (calc_action, x, 0, faculty(x)))
```

In this exercise we will not consider the Sum and Average calculation types; just remove the code that writes their data to the file. We will however consider the Add, Subtract, Divide and Multiply calculation types. For these replace the file access code with the following database access code, for instance for the Add calculation type :

```
with con:
    cur = con.cursor()
    cur.execute("INSERT INTO Calculations ('Id', 'Nr1', 'Nr2',
'Result') VALUES(?,?,?,?)",
                (calc_action, f1, f2, add(f1, f2)))
```

Next insert some records in the database, observe the existence of the `calculations.db` file in the directory of the script and use the SQLiteStudio Database Viewer to connect to SQLite and examine the contents of the database.

Also make a script to read the database records back in and display them in the console.

Exercise 11 : Python Classes

In this exercise you build an Employee class and a Company script around it.

Step 1 : Employee class

Make an `Employee` class representing some employee working in a company in the Python module `employee.py`. The `Employee` class should have two attributes representing the name and the ID of an employee.

Add a constructor to the `Employee` class which sets the attributes when they are passed. Make a simple program which declares an `Employee` object and displays the values of the attributes of the `Employee` object by accessing the attributes directly :

```
class Employee:
    def __init__(self, name):
        print ("Calling Employee constructor")
        self.name = name
```

Next make the attributes private in the `Employee` class and add public set and get member functions to be able to access these attributes. Change the values of the attributes through the accessor functions and display their changed values :

```
def set_name(self, value):
    self.__name = value

def del_name(self):
    del self.__name
```

Notice that private data cannot be accessed directly from outside of the class. This demonstrates the object oriented principle of encapsulation. In Python there is a workaround so you can still access private data; use this workaround also for demonstration purposes.

Also add a destructor to the class and keep a class variable which counts how many instances of the class are present :

```
def __del__(self):
    class_name = self.__class__.__name__
    print (class_name, "destroyed")
```

Write print statements when the constructor, destructor, getters and setters are called. Also print an instance variable directly. What is the result?

Next make a property in the class with the following code :

```
name = property(get_name, set_name, del_name, "name's docstring")
```

Access the `Employee` object through the property as follows :

```
e1.name = "Some Name"
print (e1.name)
```

Also write statements to show the class and object data :

```
print ("Employee.__doc__:", Employee.__doc__)
print ("Employee.__name__:", Employee.__name__)
print ("Employee.__module__:", Employee.__module__)
print ("Employee.__bases__:", Employee.__bases__)
print ("Employee.__dict__:", Employee.__dict__)
```

Put the `Employee` class in a script in the `employees` package and add code to be able to export the `Employee` class from this package. Give the `Employee` class a `calc_salary()` method as follows :

```
def calc_salary(self):
    return 0
```

Optionally you can also add a `hiringdate` attribute to the `Employee` class. Add the following code to test the `Employee` class in stand alone mode :

```
if __name__ == '__main__':
    main()
```

Step 2 : Company Script

Make a `Company` script above the `employees` package. Add the necessary import statements to be able to find the `Employee` class in the employee script in the `employees` package. If you keep the `__init__.py` file in the package empty your code could look as follows :

```
import employees.employee
e1 = employees.employee.Employee("Albert Einstein")
```

Create a number of employees and put them in a list, called for instance `employee_list`. Next iterate through the list and print the name of the employees in the list.

If time permits : alternatively you could create a `Company` class with functions to add or to remove employees from the `employee_list`, which displays a list of employees including their name and ID. In that case make a simple script which declares a `Company` object, add several employees to the company and display a list of employees.

The relationship between the `Company` class and the `Employee` class is an association relation of the normal type. We can say every Company employs Employees, or every Employee works for a Company.

Exercise 12 : Type Hinting APPENDIX

In this exercise you annotate code you have already written, and let a checker read the annotations for you. Work in the `course-work` project you created in Exercise 1.

Step 1 : Install the checker

`mypy` is a development dependency, so add it to the project with the `--dev` flag :

```
uv add --dev mypy
```

Step 2 : Annotate a function

Create a module `shapes12.py` with two functions and no annotations at all :

```
def area(width, height):
    return width * height


def describe(name, area):
    return f"{name} has an area of {area}"


print(describe("kitchen", area(3.0, 4.0)))
```

Run it with `uv run python shapes12.py` and confirm it works. Now annotate both functions. Every parameter takes a colon and a type, and the result takes an arrow :

```
def area(width: float, height: float) -> float:
    ...
```

Do the same for `describe`, which takes a string and a number and returns a string. Run the program again and confirm the output has not changed. Annotations do not alter what the program does.

Step 3 : Let the checker find a mistake

Run the checker over the module :

```
uv run mypy shapes12.py
```

It should report nothing. Now introduce a mistake on purpose. Change the last line so that it passes the name and the area the wrong way round :

```
print(describe(area(3.0, 4.0), "kitchen"))
```

Run the program. Notice that Python is perfectly happy and prints nonsense. Now run `uv run mypy shapes12.py` again. The checker reports the mismatch and names the argument that is wrong, without the program having run at all. Put the call back the way it was.

Step 4 : Annotate a collection

Add a function that takes a list of numbers and returns their average :

```
def average(values):
    return sum(values) / len(values)
```

Annotate it. The parameter is a list of floating point numbers, written `list[float]`, and the result is a `float`. Call it with `average([1.0, 2.0, 3.0])` and check it with `mypy`.

Then call it with a list of strings, `average(["a", "b"])`, and run `mypy` again. The checker reports the mismatch even though the offending value is inside a list.

Step 5 : A value that may be absent

Add a function that greets someone whose name you may not know :

```
def greet(name=None):
    if name is None:
        return "Hello, stranger"
    return f"Hello, {name}"
```

Annotating this as `name: str = None` would be a lie, because `None` is not a `str`. Write it as `name: str | None = None` instead, and give the function a `→ str` result. Confirm with `mypy` that both `greet()` and `greet("Alice")` are accepted.

Step 6 : Annotate a class

Take the `Employee` class from Exercise 11 and annotate its `__init__` method. The `self` parameter is never annotated. A method that returns nothing is annotated `→ None` :

```
class Employee:
    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self.salary = salary
```

Annotate the other methods as well, then run `mypy` over the module and fix whatever it reports.

If time permits : run `uv run mypy` over one of your solutions from an earlier exercise, with no annotations added at all. Notice how little it can say. Then annotate one function and watch how much further it can reason from that single signature.

Exercise 13 : Python Libraries APPENDIX

An optional exercise covering the four standard library modules from the appendix. Each step stands on its own, so take them in whatever order interests you. Work in your `course-work` project.

Step 1 : datetime

Create a module `dates13.py`. Write a function `days_until(target)` that takes a `date` and returns the number of days between today and that date. Use it to print how many days remain until the next New Year's Day.

Then print today's date in three formats :

- `15-08-2026`
- `Saturday 15 August 2026`
- the ISO form, `2026-08-15`

Finally, read the string `"24-12-2026 18:00"` back into a `datetime` with `strptime` and print the weekday it falls on.

Step 2 : json

Create a module `books13.py` holding a list of at least three dictionaries, each with a `title`, a `pages` count and a list of `tags`.

Write the list to `books.json` with `json.dump`, using `indent=2` so the file is readable. Open the file in your editor and look at it. Then read it back with `json.load` and print the title of every book with more than 500 pages.

Now add a `published` key whose value is a `date` object and try to write the file again. Read the error carefully. Fix it by converting the date to a string with `isoformat()` before writing.

Step 3 : Regular expressions

Create a module `log13.py` with the following lines as a list of strings :

```
2026-08-15 12:30 ERROR disk full
2026-08-15 12:31 INFO backup started
2026-08-15 12:45 WARN disk at 91 percent
2026-08-16 09:02 ERROR connection refused
```

Write a regular expression with named groups for the date, the time, the level and the message. For each line, print only the entries whose level is `ERROR`, formatted as `date time : message`.

Then use `re.findall` to collect every number that appears in any message, and `re.sub` to replace every date with the word `[date]`.

Step 4 : unittest

Create a module `calculator13.py` with four functions, `add`, `subtract`, `multiply` and `divide`. Make `divide` raise a `ValueError` when asked to divide by zero.

Create a second module `test_calculator13.py` with a `TestCase` class that tests all four. Include :

- a test per function for the ordinary case
- a test that `divide` raises `ValueError` on a zero divisor, using `assertRaises`
- a `setUp` method that prepares any values the tests share

Run the tests with `python -m unittest test_calculator13.py -v` and read the output. Then break one function on purpose and run the tests again, so that you see what a failure report looks like.

If time permits : the `appendices/` folder holds reference PDFs on several further libraries, and `demos/13-appendix-libraries/` holds a runnable example for every slide in the appendix module.